

Likelihood-Update Merge

Oscar Laird

April 2026

1 Overview

2 Definitions

Fix an alphabet Σ with a distinguished start symbol $^{\wedge}$. Write Σ^* for the finite strings over Σ , and write $u \preceq x$ when u is a prefix of x .

Definition 1 (Rooted likelihood update). *A rooted likelihood update U consists of:*

- *a finite set $\text{nodes}(U) \subseteq \Sigma^*$,*
- *a weight $\ell_U(n) \in \mathbb{R}$ for each $n \in \text{nodes}(U)$,*
- *the requirement that $^{\wedge} \in \text{nodes}(U)$,*
- *the requirement that $\text{nodes}(U)$ is prefix-closed.*

The Rust implementation stores an explicit leaf bit. Mathematically, this bit is redundant: a present node is a leaf exactly when it has no present children.

Definition 2 (Truncation). *Let U be a rooted likelihood update and let $x \in \Sigma^*$ extend the start symbol. Define*

$$\tau_U(x) := \max_{\preceq} \{n \in \text{nodes}(U) : n \preceq x\},$$

the longest present prefix of x in U . This is well defined because $^{\wedge} \in \text{nodes}(U)$.

Definition 3 (Lookup through truncation). *For a rooted likelihood update U and a string x , define*

$$L_U(x) := \ell_U(\tau_U(x)).$$

Definition 4 (Semantic merged lookup). *Given rooted likelihood updates $U_1, \dots, U_k$, define their semantic merged lookup by*

$$L^*(x) := \sum_{i=1}^k L_{U_i}(x).$$

This function L^* is the mathematical specification of the merge. The formal task is to show that the constructed merged update realizes this specification.

3 Construction

For each string $x \in \Sigma^*$, define

$$h(x) := \text{the longest among } \tau_{U_1}(x), \dots, \tau_{U_k}(x).$$

Since each $\tau_{U_i}(x)$ is a prefix of x , this longest string is well defined.

We define the merged rooted likelihood update M as follows.

Definition 5 (Merged node set). *A string n belongs to $\text{nodes}(M)$ if and only if there exists a string x such that*

$$n \preceq h(x)$$

and such that $n \in \text{nodes}(U_i)$ for at least one index i .

Definition 6 (Merged weights). *For each $n \in \text{nodes}(M)$, define*

$$\ell_M(n) := \sum_{i=1}^k \ell_{U_i}(\tau_{U_i}(n)).$$

Thus the weight at a merged node is defined directly from the input updates: each update contributes the weight of its own truncation point on the branch leading to n .

The merge routine `merge_many` is intended to compute exactly this mathematical object.

4 Correctness

Lemma 1 (Location of the merged truncation point). *Fix $x \in \Sigma^*$. For each i , let $\tau_i := \tau_{U_i}(x)$. Then $\tau_M(x)$ is the longest among the strings $\tau_1, \dots, \tau_k$.*

Proof. Each τ_i is a prefix of x , so the strings $\tau_1, \dots, \tau_k$ are totally ordered by the prefix relation, equivalently by length.

Fix a prefix $n \preceq x$. The update U_i has already stopped by the time the traversal reaches n if and only if $\tau_i \preceq n$. Therefore all updates have stopped by the time the traversal reaches n if and only if every $\tau_i \preceq n$.

The merged truncation point is the first prefix of x with that property. Since the τ_i are linearly ordered, that first such prefix is exactly the longest among $\tau_1, \dots, \tau_k$. $\square$

Lemma 2 (Value at the merged truncation point). *Fix $x \in \Sigma^*$ and let $h := \tau_M(x)$. Then*

$$\ell_M(h) = \sum_{i=1}^k \ell_{U_i}(\tau_{U_i}(x)).$$

Proof. Partition the index set $\{1, \dots, k\}$ into

$$I_{=} := \{i : \tau_{U_i}(x) = h\} \quad \text{and} \quad I_{<} := \{i : \tau_{U_i}(x) \prec h\}.$$

If $i \in I_{=}$, then U_i contains the node h , so its contribution appears as the summand $\ell_{U_i}(\tau_{U_i}(x)) = \ell_{U_i}(h)$.

If $i \in I_{<}$, then U_i stopped strictly above h . Its contribution at h is still the summand $\ell_{U_i}(\tau_{U_i}(x))$, now coming from a strict ancestor of h .

These two classes are disjoint and exhaustive, and each index contributes exactly once. Summing the direct and carried contributions yields

$$\ell_M(h) = \sum_{i=1}^k \ell_{U_i}(\tau_{U_i}(x)).$$

$\square$

Theorem 1 (Constructed merge realizes the specification). *For every string $x \in \Sigma^*$,*

$$L_M(x) = L^*(x).$$

Equivalently,

$$L_M(x) = \sum_{i=1}^k L_{U_i}(x).$$

Proof. By definition of lookup through truncation,

$$L_M(x) = \ell_M(\tau_M(x)).$$

Apply the previous lemma with $h = \tau_M(x)$ to obtain

$$L_M(x) = \sum_{i=1}^k \ell_{U_i}(\tau_{U_i}(x)).$$

Finally rewrite each summand as $L_{U_i}(x)$. □

5 Implementation correspondence

The preceding theorem is the mathematical statement. The Rust routine `merge_many` is intended to compute the update M just analyzed. Its running quantities match the two terms in the proof:

- `n_direct_l` accumulates the direct contributions;
- `n_proper_truncated_l` accumulates the carried contributions;
- `n_edge_hit_count == len` expresses that every input update has stopped on the current branch.

6 Next formal step

This exact theorem is the right first target for Lean. The later floating-point theorem should be stated separately: first define the exact real-valued merge semantics, then compare the `f32` implementation against that exact target with an explicit rounding-error bound.